Частина 3: стрімінг локально (Redpanda)

OOAD, лабораторна 8 · timlim24 · github.com/timlim24/OOAD-lab-8

part3-streaming-local/README.md

Частина 3 — Стрімінг локально: Redpanda + Python

Запуск

```
# 0. одноразово: доступ до docker без sudo
sudo usermod -aG docker $USER && newgrp docker

make venv          # .venv + kafka-python-ng
make data          # качає dt=2026-10-01 з публічного бакета
make up            # redpanda + console, чекає на healthy
make topic         # топік payments, 3 партиції
make produce       # реплей дня, ~20 платежів/с
make consume       # у другому терміналі
```

Console — http://localhost:8080, брокер — `localhost:19092`.

Файл	Що це
docker-compose.yml	Redpanda v25.2.5 + Console v3.4.0
producer.py	реплей дня в топік у порядку `receivedAt`
consumer.py	дедуплікація + віконна частка відмов + алерти
simulate.py	та сама логіка консюмера **без брокера** — для перевірки
Makefile	`make help`

`simulate.py` — навіщо

Імпортує той самий клас `WindowedFailureRate` із consumer.py і згодовує йому файл у порядку `receivedAt` — тобто рівно те, що надіслав би продюсер. Дає змогу перевірити логіку вікон до того, як з'явиться Docker, і — головне — **отримати очікувані числа для кожного експерименту**, щоб відрізнити реальну різницю від власної помилки.

Усі цифри нижче отримані саме ним на `dt=2026-10-01` і **повністю детерміновані**: і вибір «зламаних» платежів, і причина відмови виводяться з хешу `invoiceId`, без жодного RNG. Про те, чому це принципово, — нижче, у розділі про експеримент 3.

Як воно влаштоване

Продюсер: стискання часу

День 2026-10-01 — це 101 084 події для 33 361 платежу, розтягнутих на 24 години. Щоб віддавати 20 платежів/с, увесь день треба стиснути в `33 361 / 20 ≈ 1 668 с ≈ 28 хвилин`, тобто **speed factor ≈ 52×**: одна секунда реального часу = 52 секунди часу подій.

Важливо, що масштабується **весь** таймлайн, разом із запізненнями. Подія, яка в датасеті прийшла на 10 хвилин пізніше, прийде на `600/52 ≈ 11,5` реальних секунд пізніше. Для консюмера це прозоро, бо його вікна живуть в **event time**, а не в реальному: він бачить ті самі 60-секундні вікна й ті самі 10-хвилинні запізнення, просто все відбувається швидше.

Сон рахується від `wall_start` щоразу заново, а не накопиченням `sleep()`: інакше кожна повільна відправка зсувала б увесь подальший реплей, і до кінця дня дрейф набіг би на хвилини.

Чому `key=invoiceId`

`kafka-python` хешує ключ **murmur2** — тією ж функцією, що й Java-клієнт. Усі три події одного платежу дають той самий хеш → ту саму партицію → **той самий порядок**. Без ключа повідомлення розкладались би round-robin і `success` міг би обігнати `processing` у різних партиціях.

Порядок Kafka гарантує **тільки в межах партиції**. Ключ — це єдиний спосіб перетворити цю гарантію на щось корисне.

Консюмер: watermark і grace

```
watermark = max(modifiedDate), який ми бачили
вікно [T, T+60) закривається, коли watermark >= T + 60 + grace
подія, чиє вікно вже закрите → rejected as late
```

Watermark монотонний за побудовою (`max()` не зменшується). Це той самий підхід, що у Flink `BoundedOutOfOrderness`. Чого він не вміє — **рухатися, коли стрім мовчить**, тому в консюмері є ще й idle-flush: якщо 15 секунд нічого не прийшло, відкриті вікна закриваються примусово. Без цього останні вікна скінченного реплею не закрилися б ніколи, і консюмер виглядав би зависнутим.

Знаменник — тільки термінальні події

`created` і `processing` — це не «не відмова», це «відповіді ще немає». Якщо покласти їх у знаменник, отримаємо рівно ту помилку, яку виміряно в частині 2: **6 % перетворюється на 2 %**. Тому консюмер рахує частку по `success` + `failure`, а решту лише лічить окремо (`non-terminal (skipped)`).

Ключ дедуплікації

`(invoiceId, status, modifiedDate)` — **бізнес-ідентичність** події, а не хеш байтів. Та сама (платіж, статус, момент) — це та сама подія, навіть якщо ретрай переserialised JSON з іншим порядком ключів чи пробілами. Хеш байтів такий ретрай пропустив би.

Базовий прогін

Продюсер відпрацював повний день у топік `payments` рівно за планом:

```
sent        : 101,084 events for 33,361 payments
delivered   : 101,084
wall time   : 1668.1s (20.0 payments/s)
```

Рівно **20,0 платежів/с** — цільовий темп із завдання. Темп нерівний: на початку реплею (нічні години) ~13/с, під кінець (вечірній пік) ~21/с, бо реплей зберігає форму добового профілю.

Консюмер на брокері (3 партиції) і офлайн-еталон на тому самому файлі:

	`simulate.py` (без брокера)	**консюмер на Redpanda**
events received	101 084	**101 084**
duplicates dropped	1 001 (0,990 %)	**1 001 (0,990 %)**
rejected as LATE	1 680 (1,662 %)	140 (0,138 %)
counted into windows	32 798	33 314
windows closed	1 433	**1 433**
alerts fired	51	55
dedup set, пік	100 083	**100 083**

Кількість подій, дублікатів, вікон і розмір множини дедупу збігаються **точно**. Різниця лише в запізнілих — і вона пояснюється нижче, в експерименті 4: poll-батч працює як додатковий grace.

Звірка з батчем: 0,990 % дублікатів проти 0,997 % з Athena по всьому місяцю. 1 433 вікна замість 1 440 — сім хвилин доби не мали жодної термінальної події (глибока ніч).

Експеримент 1 — зупинити консюмер на хвилину

```
make produce            # хай крутиться
make consume            # Ctrl-C через 2 хвилини
sleep 60
make consume            # запустити знову
```

Що відбувається. Нічого не губиться. Kafka — це **лог**, а не черга: повідомлення не зникає після читання, воно лежить у партиції до закінчення retention. Консюмер зберігає в брокері лише **offset** — номер позиції у кожній партиції. При рестарті та сама `group.id` отримує свої офсети назад і читає далі з того самого місця.

У консюмері навмисно `enable_auto_commit=False` і ручний `commit()` раз на 5 секунд **після** того, як події згорнуті у вікна. З автокомітом клієнт міг би закомітити офсет повідомлення, яке прочитав, але ще не обробив, — і падіння між цими двома моментами втратило б дані. Ручний коміт дає **at-least-once**: після рестарту до 5 секунд подій прийдуть удруге, і їх

прибере та сама дедуплікація, що й ретраї вебхука. Те саме колесо, два різні джерела дублікатів.

Що подивитися в Console: вкладка *Consumer Groups* → `failure-rate-monitor`. Поки консюмер стоїть, **Lag** росте — це буквально кількість повідомлень, які його чекають. Після рестарту lag за кілька секунд падає до нуля: консюмер «доганяє» вичитуванням з диска швидше, ніж продюсер пише.

На що звернути увагу у виводі. Стан вікон у пам'яті при зупинці **втрачається** — відновлюються тільки офсети. Якщо вікно було наполовину зібране, після рестарту воно перечитається з офсету й збереться наново; але якщо коміт устиг пройти за середину вікна, частина подій уже не перечитається і вікно закриється неповним. Це головна причина, чому Flink має **чекпоінти стану**, а не лише офсети: у проді зберігати треба і те, і те, атомарно.

> Повний вивід і таблиця lag: results/exp1_restart.md

Експеримент 2 — два консюмери в одній групі

```
make consume            # термінал 1
make consume            # термінал 2, та сама group.id
```

Що відбувається. Топік має 3 партиції, консюмерів стає 2 → брокер запускає **rebalance** і роздає партиції: одному дві, іншому одну. Обидва консюмери друкують це самі:

```
[rebalance] revoked:  [0, 1, 2]
[rebalance] assigned: [0, 1]
```

```
[rebalance] assigned: [2]
```

У Console: *Consumer Groups* → `failure-rate-monitor` показує два members і навпроти кожного — його партиції та окремий lag.

Три речі, які варто сказати на захисті:

1. **Партиція — це одиниця паралелізму.** Трьох консюмерів вистачить, щоб узяти по одній партиції. **Четвертий отримає нуль партицій і простоюватиме** — додавання консюмерів понад кількість партицій не прискорює нічого. Кількість партицій — це стеля масштабування, і міняти її потім боляче.
2. **Результати стають неповними, і це очікувано.** Кожен консюмер рахує вікна **тільки по своїх партиціях**, тож замість однієї частки відмов по `card` ви отримаєте два частинні підрахунки в двох терміналах. Щоб зводити глобально, потрібен або один консюмер, або другий етап агрегації (репартиція за `paymentMethod` — саме це робить Flink `keyBy`). Наш ключ — `invoiceId`, тож платіж не роз'їжджається, а `paymentMethod` роз'їжджається.
3. **Під час rebalance обробка стоїть.** З дефолтним eager-протоколом усі консюмери віддають усі партиції й отримують нові — «stop the world» на секунди. Саме тому в

Kafka з’явився cooperative rebalancing.

> Повний вивід, склад групи і випадок із 4 консюмерами: results/exp2_two_consumers.md

---

Експеримент 3 — 30 % відмов

```
make produce ARGS="--fail-rate 0.30"
# або: python producer.py --file data/events-2026-10-01.jsonl --fail-rate 0.30
```

Продюсер детерміновано (за хешем `invoiceId`) переписує 30 % платежів у `failure`, тримаючи запис узгодженим: `finalAmount = 0` і з’являється `failureReason`.

Підводний камінь, на який я наступив

Перша версія брала причину відмови через `rng.choice(FAILURE_REASONS)`. Виглядає нешкідливо — і тихо ламає саме те, що лабораторна вимірює.

У датасеті ~1 % подій — **байт-у-байт однакові передоставлення**. Коли продюсер переписує таку подію у `failure`, він робить це **двічі** — для оригіналу й для копії. З `rng.choice` два послідовні виклики дають **різні** причини, і дві копії перестають бути дублікатами.

Виміряно на Kinesis-прогоні через Athena:

	дублікатів у дні
датасет як є	**1 001**
після `--fail-rate 0.30` з `rng.choice`	**920**

Ін'єкція відмов знищила 8 % дублікатів — тобто рівно той сигнал, який дедуплікація має ловити. Виправлення: причина виводиться з `sha256("reason:" + invoiceId)`, тож усі копії однієї події отримують однакову причину й лишаються ідентичними. Після виправлення `duplicates dropped` дорівнює 1 001 і в базовому режимі, і з `--fail-rate`.

Мораль ширша за цю лабораторну: **генератор тестових даних, який підмішує випадковість на рівні події, непомітно змінює статистичні властивості датасету.** Усе, що має бути однаковим у копіях, мусить виводитися з ключа, а не з RNG.

Результат на брокері: алертів **55 → 326**, у 5,9 раза більше. Решта лічильників не змінюється — 101 084 події, 1 001 дублікат, 1 433 вікна в обох прогонах, бо ми міняємо лише значення поля `status`, а не час і не доставку. Офлайн-еталон дає 51 → 304, той самий множник. Деталі: results/exp3_failures.md.

Але головне тут інше: 51 алерт на базовому прогоні — це хибні спрацювання

Справжня частка відмов у даних — **6 %**, поріг — **10 %**. Звідки 51 алерт? З маленьких вікон. Уночі в хвилинному вікні буває 2–6 термінальних подій; одна відмова з трьох — це 33 %, і поріг пробито шумом.

Тому в консюмері є `--min-events` (за замовчуванням 20): не алертити на комірку, менша за N. Ось як вибір цього порога впливає на розрізнювальну здатність — виміряно на тому самому дні:

`--min-events`	комірок ≥ N	алертів при 6 % (хибні)	алертів при 30 %	розділення
1	4 430	909 (**20,5 %**)	3 437 (77,6 %)	погане
5	2 498	662 (**26,5 %**)	2 373 (95,0 %)	погане
10	1 264	234 (**18,5 %**)	1 224 (96,8 %)	середнє
20	**307**	**51 (16,6 %)**	**304 (99,0 %)**	**найкраще**

Без захисту за розміром вибірки монітор кричить на кожній п’ятій нормальній комірці — і його вимкнуть першого ж тижня. З `min-events = 20` хибні спрацювання падають до 16,6 %, а при реальній аварії алерт ловить 99,0 % комірок. Це і є **чутливість проти специфічності**: поріг у відсотках без мінімального розміру вибірки — не монітор, а генератор шуму.

Дорослий варіант — не фіксований поріг, а **біноміальний тест** проти базової частки 6 % (або контрольний ліміт $\hat{p} \pm 3\sigma$), який автоматично вимагає більшої аномалії від меншої вибірки.

> Повні цифри: results/exp3_failures.md

Експеримент 4 — grace = 0

```
python simulate.py --file data/events-2026-10-01.jsonl --grace 0
# з брокером: python consumer.py --grace 0
```

Обидва консюмери читають **той самий повний лог** із початку, різними групами, відрізняються лише `--grace`:

	`--grace 120`	`--grace 0`	різниця
events received	101 084	101 084	—
duplicates dropped	1 001 (0,990 %)	1 001 (0,990 %)	—
rejected as LATE	**140 (0,138 %)**	**227 (0,225 %)**	**+87, +62 %**
наскільки запізнилися (сер.)	163 с	190 с	+27 с
наскільки запізнилися (макс.)	427 с	537 с	+110 с
counted into windows	**33 314**	**33 284**	**−30 термінальних**
non-terminal (skipped)	66 629	66 572	−57
windows closed	1 433	1 433	—
alerts fired	55	53	−2

Офлайн-еталон на тому самому файлі дає 1 680 → 2 532 (+50,7 %) і −299 термінальних. Напрямок і співвідношення ті самі, масштаб інший — чому саме, нижче.

Що саме змінилося в числах

Відкинутих як запізнілі стало в 1,6 раза більше. З grace = 120 вікно чекає ще дві хвилини після свого кінця, і все, що прийшло за цей час, ще встигає. З grace = 0 воно закривається, щойно watermark перетнув кінець.

Вікна стали неповними: −30 термінальних подій. Кількість вікон не змінилася (1 433), бо вікно створює перша подія, що в нього потрапила, а не grace.

Максимальне зафіксоване запізнення зросло з 427 до 537 с — при нульовому grace консюмер «бачить» запізнення, які зі 120-секундним запасом просто не були б запізненнями.

Чому на брокері цифри менші, ніж в офлайн-еталоні

Це не помилка, і причину варто назвати: **розмір батчу працює як додатковий, непроханий grace.**

- `simulate.py` викликає `close_ready()` **після кожної події** — вікна закриваються за найтіснішим можливим графіком.
- Консюмер викликає `close_ready()` **раз на poll-батч** (до 500 записів). Поки батч обробляється, watermark стоїть, і всі запізнілі події всередині батчу встигають у свої вікна.

Коли консюмер наздоганяє лог із диска, один батч покриває кілька хвилин часу подій, тож фактичний grace дорівнює `grace + (проміжок часу подій в одному батчі)`, і другий доданок переважає перший. На живому стрімі, де батч збирається за сотні мілісекунд, цифри зійшлися б із симуляцією.

Сама по собі корисна річ: **латентність обробки впливає на результат віконних обчислень так само, як налаштування grace.** Два консюмери з однаковим `--grace`, але різною швидкістю читання, дадуть різні числа.

Частки відмов зсунулися несильно (55 проти 53 алертів) — але це властивість датасету, а не загальне правило. І ось найважливіше: запізнілі події **не випадкові**. У цьому датасеті затримку отримують 2 % подій випадково, тож зсув малий. У проді запізнюється те, що проходило складніший шлях: 3-D Secure з таймаутом, ретраї після збою банку — тобто **непропорційно багато відмов**. Викинувши запізнілі, монітор систематично **занижує** частку відмов саме тоді, коли вона росте, — тобто сліпне рівно в момент аварії.

Кількість вікон не змінилася (1 433), бо вікно створюється першою подією, що в нього потрапила, а не grace-ом.

Компроміс

grace	покриття подій	затримка алерту
0 с	32,65 % на момент закриття	миттєво
120 с	98,22 %	+2 хв
600 с	100,00 %	+10 хв

Купити останні 1,8 % повноти коштує ще **вісім хвилин** затримки на кожному алерті. Для моніторингу платежів 2 хвилини — правильний вибір: алерт через 10 хвилин після початку аварії марний. Для нічного звіту правильний вибір — 600 секунд, бо там затримка нічого не коштує, а повнота коштує всього.

Це і є різниця між стрімом і батчем в одному реченні: батч бере grace, що дорівнює добі, і тому завжди правий; стрім бере grace у дві хвилини і тому завжди трохи неправий — але вчасно.

> Повні цифри: results/exp4_grace_zero.md

Множина побачених подій росте без обмежень

Після одного дня `dedup set` містить **100 083** ключів — усі унікальні події доби. За місяць це були б 3 000 000, а в масштабах Uklon з 580 млн GPS-пінгів на добу — неможливо в принципі.

Кожен ключ — кортеж із трьох рядків (`invoiceId`, `status`, `modifiedDate`). У CPython це близько 200 байтів з урахуванням накладних витрат dict → **~20 МБ за добу**, ~600 МБ за місяць, і процес помирає по OOM. Не «колись», а за календарем.

Що з цим роблять у проді:

Підхід	Як	Чим платимо
Redis з TTL	`SET dedup:<key> 1 EX 900 NX`	мережевий похід на кожну подію; дедуплікація стає зовнішньою залежністю
Flink keyed state + state TTL	`StateTtlConfig`, стан у RocksDB, шардований за ключем	складність Flink; зате стан локальний і чекпоінтиться разом з офсетами
Bloom filter з ротацією	два фільтри по 15 хв, старий викидається	можливі хибнопозитивні → **втрата реальних подій**
Ідемпотентний стік	не дедуплікувати взагалі, писати `UPSERT` по ключу	потрібне сховище з первинним ключем; не працює для лічильників

У консюмері реалізовано перший варіант у пам’яті: `--dedup-ttl N` викидає ключі, старші за N секунд **часу подій**:

```
python simulate.py --file data/events-2026-10-01.jsonl --dedup-ttl 900
```

Виміряно на тому самому дні:

	без TTL	`--dedup-ttl 900`
duplicates dropped	1 001 (0,990 %)	**1 001 (0,990 %)**
dedup set, пік	100 083	**2 202**
dedup set, у кінці	100 083	337
пам’ять (оцінка)	~20 МБ	**~0,4 МБ**

Дедуплікація не втратила жодного дубліката, а пам’ять впала у 45 разів. Це безкоштовно саме тому, що TTL (900 с) із запасом перекриває максимальне запізнення в датасеті (600 с): ретрай фізично не може прийти пізніше, ніж живе його ключ.

TTL має перевищувати максимальне запізнення, проти якого ви дедуплікуєте. У цьому датасеті максимум — 600 с, тож 900 с дає запас. Поставите 300 — і ретрай, що прийшов через 8 хвилин, пройде як нова подія: дедуплікація почне мовчки пропускати дублікати саме на тих подіях, які найважче відстежити. Перевіряється тривіально — `--dedup-ttl 300` і дивитися, чи `duplicates dropped` досі 1 001.

> Головне, що тут варто зрозуміти: **дедуплікація без TTL — це не «поки що простий варіант», це витік пам’яті з відкладеним падінням.** А TTL перетворює абсолютну гарантію («жодного дубліката ніколи») на умовну («жодного дубліката в межах вікна N»), і цей компроміс треба обирати свідомо, а не виявляти в проді.

Експеримент 1 — зупинка консюмера на хвилину

Топік `payments`, 3 партиції, продюсер пише ~20 платежів/с увесь час. Група `exp1b`, дефолтні налаштування (`window=60s grace=120s`).

Прогін 1 — 50 секунд, потім SIGINT

```
  events received        : 21,642
  duplicates dropped     : 225 (1.040%)
  rejected as LATE       : 80 (0.370%)
  windows closed         : 571
```

Закомічений офсет після зупинки: **21 642**.

Lag, поки консюмер стояв

момент	p0	p1	p2	log_end	TOTAL LAG
одразу після зупинки	—	—	—	21 657	~15
через ~120 с простою	1 888	1 772	1 902	27 204	**5 562**

Продюсер за дві хвилини простою написав ще 5 562 повідомлення. Жодне з них нікуди не зникло — вони просто лежать у лозі й чекають.

Прогін 2 — рестарт

```
[rebalance] assigned: [0, 1, 2]

  events received        :  8,912
  duplicates dropped     :     86 (0.965%)
  rejected as LATE       :     53 (0.595%)
  counted into windows   :  2,922
  windows closed         :    108
```

Lag через 45 секунд після рестарту:

p0	p1	p2	committed	log_end	TOTAL LAG
7	6	8	30 554	30 575	**21**

Перевірка: нічого не втрачено

```
прогін 1 :  21 642 події
прогін 2 :   8 912 подій
  разом  :  30 554
committed:  30 554   ✔ точний збіг
```

Консюмер дочитав рівно до закоміченого офсета, без розривів і без перескоків. Lag упав із 5 562 до 21 за 45 секунд — тобто **консюмер наздоганяє швидше, ніж продюсер пише**: читання з диска послідовне й не обмежене темпом реплею.

Чому так працює

Kafka — це **лог, а не черга**. Повідомлення не зникає після читання; воно лежить у партиції до кінця retention (у нас дефолтний). Консюмер зберігає в брокері лише **offset** — позицію в кожній партиції. Та сама `group.id` отримує свої офсети назад і читає далі.

У консюмері навмисно `enable_auto_commit=False` і ручний `commit()` раз на 5 секунд **після** того, як події згорнуті у вікна. З автокомітом клієнт міг би закомітити офсет повідомлення, яке прочитав, але ще не обробив, — і падіння між цими моментами втратило б дані.

Ціна ручного коміту — **at-least-once**: до 5 секунд подій після рестарту прийдуть удруге. Видно у цифрах: `duplicates dropped` у другому прогоні 0,965 % — приблизно той самий рівень, що й у першому (1,040 %), бо перечитаних подій небагато відносно 8 912. Прибирає їх та сама дедуплікація, що й ретраї вебхука: **одне колесо на два різні джерела дублікатів**.

Що все-таки втрачається

Стан вікон у пам’яті. Відновлюються тільки офсети. Якщо вікно було наполовину зібране, після рестарту воно перечитається з офсета й збереться наново — але якщо коміт устиг пройти за середину вікна, частина подій уже не перечитається, і вікно закриється неповним. Видно у виводі: другий прогін закрив 108 вікон, причому перші з них неповні.

Це головна причина, чому Flink має **чекпоінти стану**, а не лише офсети: у проді зберігати треба і офсети, і стан, і **атомарно**. Інакше після кожного рестарту в даних лишається шрам завширшки в одне вікно.

Експеримент 2 — кілька консюмерів в одній групі

Топік `payments`, **3 партиції**.

Два консюмери, група `exp2`

Консюмер 1 стартує сам, через 25 с приєднується консюмер 2.

Вивід консюмера 1:

```
[rebalance] assigned: [0, 1, 2]     <- сам, забрав усе
[rebalance] revoked:  [0, 1, 2]     <- приєднався другий: віддав усе
[rebalance] assigned: [0, 1]        <- і отримав назад дві партиції
```

Вивід консюмера 2:

```
[rebalance] assigned: [2]
```

Стан групи в брокері під час роботи обох:

```
BALANCER   range
MEMBERS    2
TOTAL-LAG  471

TOPIC     PARTITION  CURRENT-OFFSET  LOG-END-OFFSET  LAG  MEMBER-ID
payments  0          11785           11942           157  kafka-python-…-37b2dd56
payments  1          11748           11933           185  kafka-python-…-37b2dd56
payments  2          11824           11953           129  kafka-python-…-938a8228
```

Партиції 0 і 1 належать одному member-id, партиція 2 — іншому. Розподіл 2+1.

Скільки кожен обробив:

	events received	counted into windows	windows closed
консюмер 1	37 096	12 158	753
консюмер 2	1 494	492	40

Різниця величезна, бо консюмер 1 стартував першим і встиг вичитати весь лог з початку; консюмер 2 підхопив лише свою партицію з поточної позиції групи.

Чотири консюмери на три партиції

MEMBERS	**4**
партиції мають власників	3
різних member-id серед власників	**3**

Четвертий member у групі є, але **жодної партиції не має і не обробляє нічого**.

Три висновки

1. Партиція — одиниця паралелізму. Трьох консюмерів досить, щоб узяти по одній. Четвертий простоює: Kafka ніколи не віддає одну партицію двом консюмерам однієї групи, бо це зламало б гарантію порядку. **Кількість партицій — це стеля масштабування**, і задається вона при створенні топіка. Збільшити можна, але це перетасує розподіл ключів по партиціях, тобто зламає впорядкованість для вже існуючих ключів.

2. Результати стають частинними, і це не баг. Кожен консюмер рахує вікна **лише по своїх партиціях**: замість однієї частки відмов по `card` ви отримуєте два окремі підрахунки у двох терміналах. Наш ключ — `invoiceId`, тож платіж цілком лежить в одній партиції й не роз'їжджається. А от `paymentMethod` роз'їжджається: події з `card` рівномірно розкидані по всіх трьох партиціях.

Щоб звести глобально, потрібен або один консюмер, або **другий етап з репартицією за `paymentMethod`** — саме це робить `keyBy` у Flink, і саме тому у Flink є shuffle між операторами. Наш однопроцесний консюмер цього етапу не має, і його масштабування впирається не в партиції, а в те, що агрегат за методом оплати глобальний.

3. Під час rebalance обробка стоїть. У виводі консюмера 1 видно `revoked: [0, 1, 2]` перед `assigned: [0, 1]` — він віддав **усі** партиції, включно з тими, які зараз же отримав назад. Це eager-протокол: «stop the world» на секунди при кожній зміні складу групи. Саме через це в Kafka з'явився cooperative rebalancing, який забирає лише ті партиції, що реально переїжджають.

Експеримент 3 — 30 % відмов

Щоб базовий і «зламаний» прогони можна було порівняти, вони лежать у **різних топіках**, обидва по 3 партиції, обидва — повний день `dt=2026-10-01`.

```
# базовий  -> топік payments        (група exp4-g120)
python producer.py --file data/events-2026-10-01.jsonl --rate 20

# зламаний -> топік payments-broken (група exp3)
python producer.py --file data/events-2026-10-01.jsonl \
       --topic payments-broken --rate 200 --fail-rate 0.30
python consumer.py --topic payments-broken --group exp3 --exit-after-idle 20
```

Продюсер: `101 084 events for 33 361 payments`, `forced to failure: 9 576 payments (28.7%)`, 166,8 с, рівно 200,0 платежів/с.

Результат на брокері

	базовий (6 %)	**`--fail-rate 0.30`**	**різниця**
events received	101 084	101 084	—
duplicates dropped	1 001 (0,990 %)	**1 001 (0,990 %)**	—
rejected as LATE	140 (0,138 %)	151 (0,149 %)	+11
counted into windows	33 314	33 312	−2
windows closed	1 433	1 433	—
alerts fired	**55**	**326**	**×5,9**

Усе, крім алертів, лишилося тим самим — і так і має бути: ми міняємо лише значення поля `status`, не час подій і не доставку. Офлайн-еталон (simulate.py) дає 51 → 304, тобто той самий множник.

Приклад закритого вікна зі зламаного прогону:

```
[window 21:13:00–21:14:00 UTC]  30 terminal, 9 failed, 30.00% overall
    card             13 terminal      3 failed   23.08%
    apple_pay         9 terminal      4 failed   44.44%
    google_pay        8 terminal      2 failed   25.00%
```

Підводний камінь: випадковість ламає дублікати

Перша версія брала причину відмови через `rng.choice(FAILURE_REASONS)`. Виглядає нешкідливо — і тихо псує саме те, що лабораторна вимірює.

У датасеті ~1 % подій — **байт-у-байт однакові передоставлення**. Коли продюсер переписує таку подію у `failure`, він робить це **двічі**: для оригіналу і для копії. Два послідовні виклики `rng.choice` дають **різні** причини — і копії перестають бути дублікатами.

Виміряно через Athena на Kinesis-прогоні:

	дублікатів у дні
датасет як є	**1 001**
`--fail-rate 0.30` з `rng.choice`	**920**

Ін'єкція відмов знищила 8 % дублікатів — рівно той сигнал, який дедуплікація має ловити. Виправлено: причина виводиться з `sha256("reason:" + invoiceId)`, тож усі копії однієї події дістають однакову причину. У таблиці вище `duplicates dropped` тепер 1 001 в обох режимах — це і є доказ, що виправлення працює.

Мораль ширша за лабораторну: **генератор тестових даних, який підмішує випадковість на рівні події, непомітно змінює статистичні властивості датасету.** Усе, що має бути однаковим у копіях, мусить виводитися з ключа.

---

55 алертів на базовому прогоні — це хибні спрацювання

Справжня частка відмов у даних — **6,02 %**, поріг — **10 %**. Звідки 55 алертів? З маленьких вікон: уночі в хвилинному вікні буває 2–6 термінальних подій, і одна відмова з трьох — це 33 %.

Тому в консюмері є `--min-events` (за замовчуванням 20). Як вибір цього порога впливає на розрізнювальну здатність (виміряно simulate.py на тому самому дні):

`--min-events`	комірок ≥ N	хибних при 6 %	алертів при 30 %
1	4 430	909 (**20,5 %**)	3 437 (77,6 %)
5	2 498	662 (**26,5 %**)	2 373 (95,0 %)
10	1 264	234 (**18,5 %**)	1 224 (96,8 %)
20	**307**	**51 (16,6 %)**	**304 (99,0 %)**

Без захисту за розміром вибірки монітор кричить на кожній п’ятій нормальній комірці — і його вимкнуть першого ж тижня. З `min-events = 20` хибні спрацювання падають до 16,6 %, а при реальній аварії алерт ловить 99,0 % комірок.

Це класична **чутливість проти специфічності**: поріг у відсотках без мінімального розміру вибірки — не монітор, а генератор шуму. Дорослий варіант — не фіксований поріг, а **біноміальний тест** проти базової частки 6 % (або контрольний ліміт $\hat{p} \pm 3\sigma$), який автоматично вимагає більшої аномалії від меншої вибірки.

Експеримент 4 — grace = 0

Повний день `dt=2026-10-01` уже лежить у топіку `payments` (101 084 події, 3 партиції). Два консюмери з різними групами читають **той самий лог із початку**, відрізняються лише `--grace`.

```
python consumer.py --group exp4-g120 --grace 120 --exit-after-idle 20
python consumer.py --group exp4-g0   --grace 0   --exit-after-idle 20
```

Результат на брокері

	--grace 120	**--grace 0**	**різниця**
events received	101 084	101 084	—
duplicates dropped	1 001 (0,990 %)	1 001 (0,990 %)	—
rejected as LATE	**140 (0,138 %)**	**227 (0,225 %)**	**+87, +62 %**
late: середнє запізнення	163 с	190 с	+27 с
late: максимум	427 с	537 с	+110 с
counted into windows	**33 314**	**33 284**	**−30 термінальних**
non-terminal (skipped)	66 629	66 572	−57
windows closed	1 433	1 433	—
alerts fired	55	53	−2

Обидва консюмери прочитали **рівно 101 084** події й знайшли **рівно 1 001** дублікат — стільки ж, скільки їх фізично є у файлі (`count(*) - count(DISTINCT *)` по `day01.jsonl` = 1 001). Нічого не загублено й нічого не вигадано.

Що змінилося

Запізнілих відкинуто в 1,6 раза більше. З grace = 120 вікно чекає ще дві хвилини після свого кінця. З grace = 0 воно закривається, щойно watermark перетнув кінець, — і хвіст із 1–10-хвилинних запізнень не встигає.

Вікна стали неповними: −30 термінальних подій. Кількість вікон при цьому не змінилася (1 433), бо вікно створює перша подія, що в нього потрапила, а не grace.

Максимальне зафіксоване запізнення зросло з 427 до 537 секунд — при нульовому grace консюмер «бачить» запізнення, які зі 120-секундним запасом просто не були б запізненнями.

Чому цифри менші, ніж в офлайн-еталоні

simulate.py на тому самому файлі давав 1 680 (1,662 %) і 2 532 (2,505 %). На брокері — 140 (0,138 %) і 227 (0,225 %), тобто на порядок менше. Це не помилка, і причину варто назвати:

розмір батчу працює як додатковий, непроханий grace.

- `simulate.py` викликає `close_ready()` **після кожної події**, тож вікна закриваються за найтіснішим можливим графіком.
- Консюмер викликає `close_ready()` **раз на poll-батч** (до 500 записів). Поки батч обробляється, watermark не рухається, і всі запізнілі події всередині батчу встигають потрапити у свої вікна до закриття.

Коли консюмер **наздоганяє лог із диска**, один батч із 500 записів покриває кілька хвилин часу подій — тобто фактичний grace дорівнює `grace + (проміжок часу подій в одному батчі)`, і при реплеї з диска другий доданок переважає перший. На живому стрімі, де батч збирається за сотні мілісекунд, цього ефекту майже немає, і цифри зійшлися б із симуляцією.

Напрямок і співвідношення зберігаються (+62 % відкинутих, менше подій у вікнах), тому висновок експерименту не змінюється — змінюється лише масштаб. І це сама по собі корисна річ: **латентність обробки впливає на результат віконних обчислень так само, як налаштування grace.** Два консюмери з однаковим `--grace`, але різною швидкістю читання, дадуть різні числа.

Компроміс повнота/затримка

Порахований по датасету розподіл запізнення (частина 2):

grace	покриття подій на момент закриття	затримка алерту
0 с	32,65 %	миттєво
30 с	97,99 %	+30 с
120 с	**98,22 %**	**+2 хв**
300 с	98,88 %	+5 хв
600 с	100,00 %	+10 хв

Останні 1,8 % повноти коштують **ще вісім хвилин** затримки на кожному алерті.

Найважливіше: запізнілі події не випадкові

У цьому датасеті затримку отримують 2 % подій **випадково**, тож зсув часток малий (55 алертів проти 53). У проді запізнюється те, що проходило складніший шлях: таймаут 3-D Secure, ретрай після збою банку, повторна авторизація — тобто **непропорційно багато відмов**.

Отже нульовий grace систематично **занижує** частку відмов саме тоді, коли вона росте: монітор сліпне рівно в момент аварії. Це не теоретичне застереження — це пояснює, чому поріг у 10 % із grace = 0 може не спрацювати на аварії, яка в реальності дала 25 %.

В одному реченні

Батч бере grace завдовжки в добу і тому завжди правий; стрім бере дві хвилини і тому завжди трохи неправий — але вчасно.